

# ASSIGNMENT-2.2

---

HALL NO: 2303A51215

BATCH NO: 04

Task 1: Cleaning Sensor Data

❖ Scenario:

❖ You are cleaning IoT sensor data where negative values are invalid.

❖ Task:

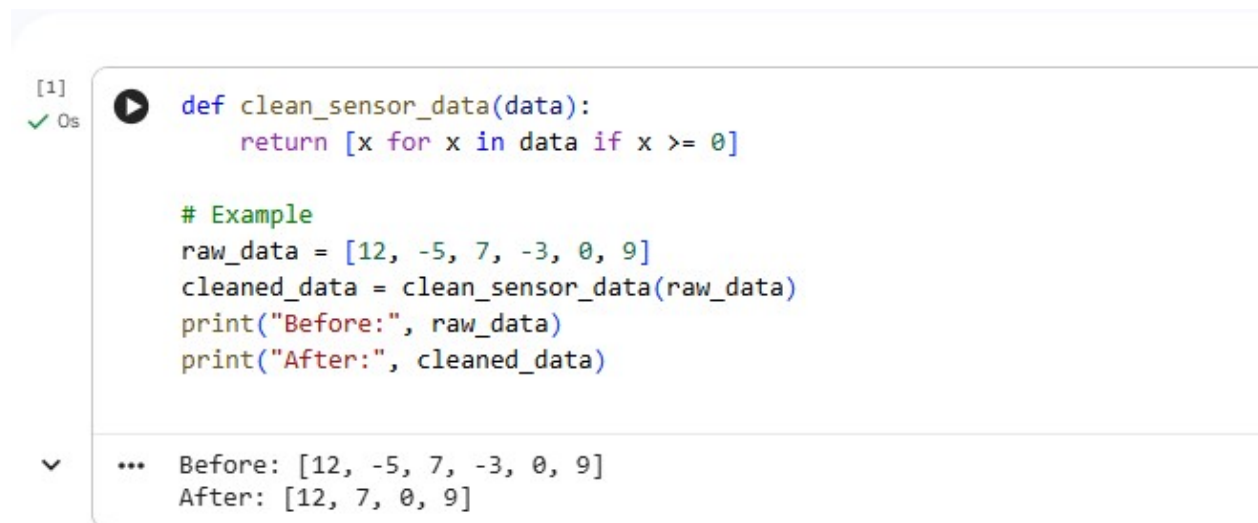
Use Gemini in Colab to generate a function that filters out all negative numbers from a list.

❖ Expected Output:

➤ Before/after list

➤ Screenshot of Colab execution

---



```
[1]
✓ 0s
def clean_sensor_data(data):
    return [x for x in data if x >= 0]

# Example
raw_data = [12, -5, 7, -3, 0, 9]
cleaned_data = clean_sensor_data(raw_data)
print("Before:", raw_data)
print("After:", cleaned_data)
```

... Before: [12, -5, 7, -3, 0, 9]  
After: [12, 7, 0, 9]

Task 2: String Character Analysis

❖ Scenario:

You are building a text-analysis feature.

❖ Task:

Use Gemini to generate a Python function that counts vowels, consonants, and digits in a string.

❖ Expected Output:

➤ Working function

➤ Sample inputs and outputs

```
[2]
✓ Os
def analyze_string(s):
    vowels = "aeiouAEIOU"
    vowel_count = sum(1 for ch in s if ch in vowels)
    consonant_count = sum(1 for ch in s if ch.isalpha() and ch not in vowels)
    digit_count = sum(1 for ch in s if ch.isdigit())
    return vowel_count, consonant_count, digit_count

# Example
text = "Hello World 2025"
v, c, d = analyze_string(text)
print(f"Vowels: {v}, Consonants: {c}, Digits: {d}")

... Vowels: 3, Consonants: 7, Digits: 4
```

### Task 3: Palindrome Check – Tool Comparison

❖ Scenario:

You must decide which AI tool is clearer for string logic.

❖ Task:

Generate a palindrome-checking function using Gemini and Copilot, then compare the results.

❖ Expected Output:

➤ Side-by-side code comparison

➤ Observations on clarity and structure

```
def is_palindrome_copilot(s):  
    left, right = 0, len(s) - 1  
    while left < right:  
        if s[left] != s[right]:  
            return False  
        left += 1  
        right -= 1  
    return True  
  
s=input("Enter a string: ")  
  
is_palindrome_copilot(s)
```

Enter a string: ARARA  
True

i] 13m

▶

```
def is_palindrome_gemini(s):  
    return s == s[::-1]  
s=input("Enter a string: ")  
  
is_palindrome_gemini(s)
```

... Enter a string: MADAM  
True

#### OBSERVATIONS:

- GEMINI gave the easier code then COPILOT..
- Copilot's solution is longer but makes the logic easy..
- GEMINI's code is done by using slicing

#### Task 4: Code Explanation Using AI

##### ❖ Scenario:

You are reviewing unfamiliar code written by another developer.

##### ❖ Task:

Ask Gemini to explain a Python function (prime check OR palindrome check) line by line.

##### ❖ Expected Output:

- Code snippet
- AI explanation
- Student comments on understanding

[7]  
✓ 4s

```
def is_prime(n):  
    if n <= 1:  
        return False  
    for i in range(2, int(n**0.5) + 1):  
        if n % i == 0:  
            return False  
    return True  
  
n=int(input("Enter a number: "))  
  
is_prime(n)
```

▼ ... Enter a number: 101  
True

AI Explanation (example style):

if n <= 1: → Numbers less than or equal to 1 are not prime.

for i in range(2, int(n\*\*0.5) + 1): → Loop through possible divisors up to  $\sqrt{n}$ .

if n % i == 0: → If divisible, not prime.

return True → Otherwise, prime.

Student Comments on understanding::

The explanation helps me to understand code easier....

I never used this logic before in coding..

I learnt a new approach for this code by explanation